

MIRAX DOCUMENTO ARCHITETTURA COMPLETA

****Obiettivo del documento:**** spiegare in modo chiaro ogni pezzo della piattaforma Mirax, a cosa serve, dove si trova, quali dati legge/scrive e come si collega agli altri componenti.

1. VISIONE GENERALE

Mirax è una piattaforma SaaS B2B per trovare, analizzare, arricchire e gestire lead aziendali. L'utente inserisce una categoria e una città il sistema crea una ricerca, i worker Python raccolgono aziende da Google Maps, il frontend mostra i risultati, l'utente può arricchire i lead, salvarli in pipeline, usare insight AI, webhook, export e billing.

1.1 Componenti principali

Componente Dove vive A cosa serve

--- --- ---

Frontend Next.js /app, /components Interfaccia utente: dashboard, ricerca, pipeline, billing, settings

API Next.js /app/api/* Backend serverless: auth check, DB, pagamenti, enrichment, webhook

Supabase Cloud Postgres + Auth Database centrale, autenticazione, sessioni, RLS

Worker Python _mirror/worker_supabase.py su VPS Prende job da Supabase, fa scraping Google Maps e audit siti

Backend FastAPI VPS _mirror/main.py Endpoint tecnici e funzioni condivise di scraping/audit

Enrichment modules /lib/*enrichment*, /lib/*analysis* Arricchiscono lead con social, recensioni, competitor, ads, AI

OpenAI API esterna Genera insight, analisi, pitch, sintesi e opportunità

Google APIs/Maps API + scraping Fonte primaria dei lead e recensioni/competitor

Stripe/PayPal API esterne Pagamenti, abbonamenti, upgrade piano, crediti

Resend API esterna Invio email, sequenze e notifiche

Webhook utente URL configurato dall'utente Invio automatico eventi verso CRM/Zapier/Make

1.2 Schema mentale semplice

UTENTE

? usa

FRONTEND NEXT.JS

? chiama

API NEXT.JS

? leggono/scrivono

SUPABASE

? contiene job pending

WORKER PYTHON SU VPS

? raccoglie dati da

GOOGLE MAPS + SITI WEB + API ESTERNE

? salva risultati in

SUPABASE

? il frontend rilegge e mostra

DASHBOARD / LEAD DETAIL / PIPELINE / INSIGHTS

2. REPOSITORY E CARTELLE

2.1 Root progetto

Percorso progetto Mirax:

c:\Users\Simone\CascadeProjects\WEB APP CKB - Copia

Non va confuso con il progetto assicurazione.

2.2 Cartelle principali

Cartella/File Serve a

--- ---

/app Pagine Next.js App Router e API routes

/components Componenti React riutilizzabili della UI

/lib Logica applicativa server-side: enrichment, analisi, webhook, resend

/types Tipi TypeScript usati da app e moduli

/utils/supabase Client Supabase browser/server/service-role

_mirror Codice Python per scraping, audit, worker e FastAPI

/migrations Migrazioni SQL Supabase

.env.local Variabili d'ambiente locali, NON da esporre

3. INFRASTRUTTURA RUNTIME

3.1 Frontend/app web

- Tecnologia: Next.js 14 App Router.
- Deploy: Vercel.
- Dominio/sito: configurato con `_PUBLIC_SITE_URL`.
- Responsabilità UI, API serverless, auth session, chiamate DB, pagamenti, enrichment lato server.

3.2 Supabase

Supabase è il centro dati del sistema. Contiene:

- Utenti autenticati.
- Profili utente e crediti.
- Job di ricerca.
- Risultati delle ricerche.
- Pipeline CRM.
- Cache degli arricchimenti.
- Configurazioni webhook.

3.3 VPS Python

Mirax usa worker Python su VPS per attività pesanti che Vercel non può fare bene:

- Browser Playwright.
- Scraping Google Maps.
- Audit siti web.
- Scraping recensioni e competitor.
- Re-audit periodico.

I worker leggono la tabella su Supabase e prendono job usando claim atomico. Questo permette di avere più worker in parallelo senza duplicare lo stesso job.

3.4 Server worker Mirax

Server Ruolo

116.203.137.39 Server storico: backend API, worker principali, audit API

178.104.182.142 Server worker aggiuntivo: più istanze parallele per processare job utente

****Nota sicurezza:**** credenziali, password e chiavi non vanno documentate nel PDF; devono stare solo in .env/secret manager.

4. DATABASE SUPABASE SPIEGATO TABELLA PER TABELLA

4.1

****A cosa serve:**** rappresenta il profilo applicativo dell'utente autenticato.

****Campi principali:****

- : uguale all'id Supabase Auth dell'utente.
- : email utente.
- `_name`: nome mostrato nelle impostazioni.
- : azienda dell'utente.
- : crediti disponibili per cercare lead.
- `_type`: piano attuale (, , , eventualmente).
- `_customer_id`: cliente Stripe associato.
- `_subscription_id`: subscription Stripe attiva.
- `_order_id`: ultimo ordine PayPal.

****Chi la usa:****

- `/app/dashboard/layout.tsx`: legge profilo e crediti quando entri in dashboard.
- `/api/profile`: legge e aggiorna nome/azienda.
- `/api/search`: controlla e scala crediti prima di creare ricerca.
- `/api/stripe/webhook`: aggiorna piano/crediti dopo pagamento.
- `/api/paypal/capture-order`: aggiorna piano/crediti dopo pagamento PayPal.
- Dashboard billing: mostra piano e crediti.

****Interconnessione:**** Auth crea l'utente, aggiunge i dati business dell'app. Senza , l'utente può autenticarsi ma non ha crediti/piano.

4.2

****A cosa serve:**** È la coda dei job di ricerca lead.

Ogni riga È una richiesta tipo: "trova dentisti a Milano".

****Campi principali:****

- : identificativo ricerca.
- _id: proprietario della ricerca.
- : categoria cercata.
- : città/località cercata.
- : zona opzionale.
- : stato job (, , ,).
- : JSON con i lead finali.
- _at: quando È stata creata la ricerca.

****Chi scrive:****

- /api/search o server action della dashboard: crea job .
- Worker Python: cambia ? processing ? completed/error.

****Chi legge:****

- Worker Python: legge job .
- Frontend /dashboard: fa polling e legge risultati.
- Storico ricerche: mostra ricerche precedenti.

****Interconnessione:**** È il ponte principale tra Next.js e il backend Python. Next.js non fa scraping direttamente: scrive un job in ; il worker Python lo processa e salva i risultati nella stessa riga.

4.3 _pipeline

****A cosa serve:**** CRM interno di Mirax. Contiene i lead salvati dall'utente e il loro stato commerciale.

****Campi principali:****

- _name, _city, _category: dati base.
- _phone, _email, _website: contatti.
- _score: score/opportunità
- _data: snapshot JSON completo del lead arricchito.
- : stato nel funnel (, , , ,).
- _value: valore economico stimato.
- : note commerciali.

****Chi la usa:****

- /api/pipeline: CRUD lead pipeline.
- /dashboard/pipeline: kanban visuale.
- /api/pipeline/stats: metriche commerciali.
- /api/insights/ai: genera insight AI sui deal.
- Re-audit worker: può aggiornare _data se cambiano segnali tecnici.

****Interconnessione:**** un lead nasce in .results, viene arricchito in /api/enrich-lead, poi può essere copiato/salvato in _pipeline per lavorarlo commercialmente.

4.4 _cache

****A cosa serve:**** evita di rifare enrichment costosi sullo stesso dominio.

****Campi principali:****

- : dominio normalizzato, es. .it.
- : risultato completo dell'arricchimento.
- _at: data aggiornamento cache.

****Chi la usa:****

- /api/enrich-lead: prima controlla la cache; se valida, restituisce subito i dati.
- -enrichment.ts: salva enrichment fresco.

****Interconnessione:**** sta tra lead detail e moduli enrichment. Riduce chiamate API, costi OpenAI e tempi di risposta.

4.5 _webhooks

****A cosa serve:**** permette all'utente di collegare Mirax a strumenti esterni come Make,

Zapier, CRM o endpoint custom.

****Campi principali:****

- `_id`: utente proprietario.
- : URL webhook.
- : eventi abilitati.
- `_active`: webhook attivo/disattivo.

****Chi la usa:****

- `/dashboard/settings`: UI gestione webhook.
- `/api/webhooks`: CRUD configurazioni.
- `/lib/webhook.ts`: invio evento HTTP POST.

****Interconnessione:**** quando succede un evento importante, Mirax invia payload JSON al CRM esterno dell'utente.

5. AUTENTICAZIONE E SICUREZZA

5.1 Supabase Auth

Supabase gestisce login, sessione e utenti. Il frontend usa cookie/sessione Supabase per sapere chi è l'utente.

5.2 .ts

****A cosa serve:**** protegge le route private. Se l'utente non è loggato e prova ad aprire dashboard, viene mandato a `/login`.

5.3 Client Supabase

Client Dove si usa Permessi

--- --- ---

Browser client componenti client permessi utente, RLS attiva

Server client API/layout server permessi utente autenticato

Service role client API protette/worker bypass RLS, solo server

****Regola importante:**** la service role key non deve mai andare nel browser.

6. FRONTEND SPIEGATO PAGINA PER PAGINA

6.1 /

Landing page pubblica. Serve a presentare il prodotto e mandare l'utente a login/registrazione.

6.2 /login

Pagina autenticazione. Usa Supabase Auth. Dopo login, l'utente viene mandato alla dashboard.

6.3 /dashboard/layout.tsx

****A cosa serve:**** wrapper server della dashboard.

Flusso:

1. Legge utente da Supabase Auth.
2. Se manca utente, redirect a `/login`.
3. Legge per crediti e piano.
4. Se profilo non esiste, lo crea con piano free.
5. Passa dati a e .

****Interconnessione:**** collega auth, profilo, crediti e layout globale.

6.4 /dashboard

Pagina principale di ricerca. L'utente sceglie categoria, città e numero lead.

Flusso:

1. Compila form.
2. Frontend chiama API/action di ricerca.
3. Sistema controlla crediti.
4. Crea job in .
5. UI mostra stato elaborazione.
6. Polling finché worker completa.
7. Mostra tabella lead.

6.5 /dashboard/lead/[id]

Pagina dettaglio lead. Serve a vedere un singolo lead in modo approfondito.

Collegamenti:

- Prende dati dal risultato ricerca o dalla pipeline.
- Può chiamare `/api/enrich-lead`.

- Mostra social, recensioni, competitor, ads, tech audit, score.
- Permette salvataggio in pipeline.

6.6 /dashboard/pipeline

Kanban CRM. Serve a gestire lo stato commerciale dei lead.

Collegamenti:

- Legge/scrive _pipeline tramite /api/pipeline.
- Cambia stage con drag/drop o azioni UI.
- Aggiorna deal value e note.
- Alimenta gli insight AI.

6.7 /dashboard/insights

Dashboard insight. Serve a capire andamento pipeline, opportunitàe rischi.

Collegamenti:

- Chiama /api/pipeline/stats per statistiche.
- Chiama /api/insights/ai per suggerimenti AI.
- I dati arrivano da _pipeline.

6.8 /dashboard/settings

Impostazioni account.

Serve a:

- Modificare nome e azienda.
- Gestire webhook.
- Testare webhook.

6.9 /dashboard/billing

Pagina piani e pagamenti.

Serve a:

- Mostrare piano corrente e crediti.
- Scegliere piano , , .
- Pagare con Stripe o PayPal.
- Aprire portale Stripe per gestire abbonamento.

6.10 /dashboard/admin

Pannello admin. Serve per statistiche globali, gestione utenti/crediti e controllo ricerche.

Deve essere accessibile solo a profili admin.

7. API NEXT.JS SPIEGATE ENDPOINT PER ENDPOINT

7.1 Ricerca lead

/api/search

- **Serve a:** creare una nuova ricerca.
- **Legge:** utente autenticato, profilo/crediti.
- **Scrive:** con ='pending'.
- **Scala crediti:** sì, in base ai lead richiesti/ricerca.
- **Collegamento:** dopo questa API, il worker Python trova il job e lo processa.

/api/search/status

- **Serve a:** controllare se la ricerca è finita.
- **Legge:** per id ricerca.
- **Risponde:** status e risultati se completata.

/api/search/history

- **Serve a:** mostrare ricerche precedenti.
- **Legge:** filtrando _id.

7.2 Enrichment lead

/api/enrich-lead

- **Serve a:** trasformare un lead base in un lead arricchito completo.
- **Input:** lead con nome, sito, telefono, email, città categoria.
- **Flusso:**

1. Verifica utente autenticato.
2. Normalizza dominio.
3. Cerca in _cache.
4. Se cache valida, restituisce subito.
5. Se cache mancante/scaduta, chiama ().

6. Salva risultato in cache.

7. Restituisce .

****Collegamento:**** È la porta d'ingresso verso tutte le fonti di enrichment.

7.3 Pipeline CRM

/api/pipeline

****GET:**** legge i lead in pipeline.

****POST:**** aggiunge un lead alla pipeline.

****PUT:**** aggiorna stage, note, valore, dati.

****DELETE:**** rimuove un lead.

****Tabella usata:**** _pipeline.

/api/pipeline/stats

****Serve a:**** calcolare statistiche CRM: pipeline value, vinti, persi, conversione, valore medio.

7.4 Profilo

/api/profile

****GET:**** restituisce email, nome, azienda, crediti, piano.

****PUT:**** aggiorna nome e azienda.

****Tabella:**** .

7.5 Insight AI

/api/insights/ai

****Serve a:**** generare insight commerciali sulla pipeline.

****Legge:**** _pipeline.

****Invia a OpenAI:**** solo summary aggregato, non PII raw.

****Output:**** 3-5 insight con titolo, descrizione, severità e icona.

7.6 Webhook

/api/webhooks

Gestisce URL webhook dell'utente.

/api/webhooks/test

Invia un evento di prova all'URL configurato.

****Utility collegata:**** /lib/webhook.ts.

7.7 Stripe

/api/stripe/checkout

Crea sessione Stripe Checkout per piano scelto.

Collegamenti:

- Legge utente e profilo.
- Crea/riusa customer Stripe.
- Crea subscription checkout.
- Metadata contiene _user_id e _id.

/api/stripe/webhook

Riceve eventi Stripe verificando firma webhook.

Eventi gestiti:

- .session.completed: aggiorna piano, crediti, subscription id.
- .payment_succeeded: rinnova crediti mensili.
- .subscription.deleted: downgrade a free.
- .payment_failed: log errore.

/api/stripe/portal

Crea sessione portale Stripe per gestire/cancellare abbonamento.

7.8 PayPal

/api/paypal/create-order

Crea ordine PayPal per piano selezionato.

/api/paypal/capture-order

Cattura pagamento PayPal, verifica stato, aggiorna .plan_type e .credits.

7.9 Admin e cron

/api/admin/stats

Statistiche globali per admin.

/api/cron/reaudit

Endpoint chiamabile da cron per attivare re-audit periodico.

8. WORKER PYTHON E BACKEND VPS

8.1 PerchØ esiste il worker

Vercel non È adatto a browser Playwright lunghi, scraping e job pesanti. Per questo Mirax usa Python su VPS.

8.2 File principali

File Serve a

--- ---

_mirror/worker_supabase.py Polling Supabase, claim job, scraping, audit, salvataggio risultati

_mirror/main.py Funzioni core: Maps scraping, website audit, normalizzazione, FastAPI

_mirror/audit_engine.py Audit tecnico avanzato HTML/SEO/security/tech

_mirror/ckb_endpoints.py Router storico/compatibilitàper trigger scraping

8.3 Ciclo worker

1. Worker parte come servizio systemd.
2. Ogni pochi secondi legge searches dove status = pending.
3. Tenta claim atomico: pending ? processing.
4. Se claim fallisce, un altro worker ha preso il job.
5. Se claim riesce, lancia scraping.
6. Aggiorna progressivamente results mentre trova lead.
7. A fine lavoro scrive status = completed.
8. Se errore, scrive status = error con trace.

8.4 Claim atomico

Il claim atomico evita doppioni quando ci sono tanti worker:

UPDATE searches

SET status='processing'

WHERE id = job_id AND status='pending'

Solo un worker riesce a fare update. Gli altri vedono zero righe aggiornate e saltano.

8.5 Cosa raccoglie da Google Maps

Per ogni attività

- Nome azienda.
- Indirizzo.
- Telefono.
- Sito web.
- Rating.
- Numero recensioni.
- Stato scheda rivendicata.

8.6 Cosa fa l'audit sito

Se il lead ha un sito, il worker controlla:

- SSL.
- Mobile responsive.
- Meta Pixel.
- Google Tag Manager.
- TikTok Pixel.
- Google Ads/GA4.
- Chatbot.
- Sistema prenotazioni.
- E-commerce.
- SPF/DMARC.
- H1/title/SEO disaster.
- Velocità caricamento.
- Email e telefono nel sito.

8.7 Output finale worker

Il worker produce array JSON con lead pronti per il frontend:

```
- , , , , ,  
- _stack con segnali/opportunità  
- _report dettagliato.  
- _score.  
- _score.  
- _reviews.  
- _competitors.  
---
```

9. ENRICHMENT: COME UN LEAD DIVENTA COMPLETO

9.1 Orchestratore -enrichment.ts

****A cosa serve:**** È il coordinatore centrale dell'arricchimento.

Non fa tutto da solo: chiama moduli specializzati e unisce i risultati in un unico oggetto.

/api/enrich-lead

?

clayEnrichLead()

?? free/public enrichment

?? website deep scraper

?? Google reviews

?? competitor analysis

?? ads analysis

?? trends analysis

?? social analysis

?? registry analysis

?? Apollo/Snov se configurati

?? output unico ClayEnrichedLead

9.2 -enrichment.ts

Analisi gratuita del sito:

- Performance.
- Sicurezza.
- Header.
- Tracking pixel.
- CMS.
- CRM/chat.
- Booking/e-commerce.
- Schema.org.
- SEO base.
- Trigger commerciali.

9.3 -deep-scraper.ts

Scraper profondo sito ufficiale. Visita homepage e pagine tipiche come contatti, chi siamo, team, privacy.

Estrae:

- Email personali/generiche/PEC.
- Telefoni cellulari/fissi.
- Social.
- Team members.
- Partita IVA/codice fiscale.
- Indirizzo.

9.4 -enrichment.ts

Fonti pubbliche via Google search scraping:

- LinkedIn company.
- LinkedIn persone.
- Facebook/Instagram/TikTok/YouTube.
- PEC da INIPEC.
- Snippet Google descrittivi.

9.5 -enrichment.ts

Integrazione Apollo.io, se _API_KEY è configurata.

Serve a trovare:

- Persone in azienda.
- Titolo/ruolo.
- Email business.
- LinkedIn.
- Dati aziendali da dominio.

9.6 -enrichment.ts

Integrazione Snov.io, se credenziali configurate.

Serve a:

- Cercare prospect per ruolo/località
- Trovare email su dominio.
- Verificare email.
- Arricchire profilo da email.

9.7 -reviews.ts

Usa Google Places API per recuperare rating e recensioni. Poi OpenAI analizza:

- Temi positivi.
- Temi negativi.
- Sentiment.
- Opportunità commerciali.

9.8 -analysis.ts

Trova competitor locali con Google Places e usa OpenAI per:

- Competition score.
- Posizionamento mercato.
- Punti di forza/debolezza.
- Messaggio di urgenza commerciale.

9.9 -analysis.ts

Analizza presenza advertising:

- Facebook Ads Library.
- Stima Google Ads via AI.
- Budget stimato.
- Opportunità per vendere servizi ads.

9.10 -analysis.ts

Prima prova backend Hetzner con pytrends. Se non disponibile, fallback OpenAI.

Output:

- Trend crescente/stabile/in calo.
- Percentuale crescita stimata.
- Mesi di picco.
- Miglior momento per contattare.
- Opportunità mercato.

9.11 -analysis.ts

Stima presenza social e opportunità

- Instagram.
- Facebook.
- TikTok.
- LinkedIn.
- Score social.
- Piattaforme mancanti/inattive.

9.12 -analysis.ts

Profilo aziendale sintetico tramite AI:

- Anno fondazione stimato.
- Forma giuridica.
- Dipendenti.
- Fatturato qualitativo.
- Insight.

Importante: se non è sicuro, deve usare o stime qualitative.

10. SCORING E SEGNALI COMMERCIALI

10.1 Opportunity score

Serve a dire quanto il lead è interessante commercialmente.

Esempi di segnali:

- No sito web: forte opportunità
- No Meta Pixel: opportunitàads/tracking.
- No GTM/GA4: opportunitàanalytics.
- No DMARC: opportunitàemail deliverability.
- Sito lento: opportunitàperformance.
- Poche recensioni: opportunitàreputation.
- Rating basso: opportunitàreputation/marketing.
- Scheda Google non rivendicata: opportunitàlocal SEO.

10.2 Freshness score

Serve a capire quanto è recente l'audit. Più passano i giorni, più il punteggio scende.

10.3 Tech stack labels

_stack non è solo tecnologia: contiene anche assenze/opportunità ad esempio:

-, , , .

- FB PIXEL.
- GTM.
- IN SPAM (NO DMARC).
- SEO (NO H1/TITLE).
- NON RIVENDICATA.

11. BILLING, PIANI E CREDITI

11.1 Piani

Piano Crediti

--- ---:

Free/Esplora 100 una tantum lato UI; alcune API iniziano 10 se profilo nuovo

Starter 1.200/mese

Pro 3.000/mese

Agency 10.000/mese

11.2 Come funziona upgrade Stripe

/dashboard/billing

? POST /api/stripe/checkout

Stripe Checkout

? ritorno utente su /dashboard/billing?success=true

Stripe Webhook

? aggiorna profiles

piano + crediti + customer/subscription id

11.3 Come funziona upgrade PayPal

/dashboard/billing

? POST /api/paypal/create-order

PayPal approval page

? ritorno con token

POST /api/paypal/capture-order

? aggiorna profiles

piano + crediti + paypal_order_id

11.4 Come i crediti si collegano alla ricerca

- Il profilo contiene .
- La dashboard mostra crediti via .
- Quando l'utente cerca, l'API verifica crediti.
- Se insufficienti, blocca ricerca.
- Se sufficienti, crea job e scala crediti.

12. WEBHOOK E INTEGRAZIONI ESTERNE

12.1 A cosa servono

Permettono di mandare eventi Mirax a CRM e automazioni.

Esempi:

- Lead arricchito.
- Ricerca completata.

- Pipeline aggiornata.

12.2 Flusso webhook

Evento interno Mirax

?

Legge user_webhooks

?

sendToWebhook()

?

POST JSON all'URL utente

?

CRM/Make/Zapier riceve evento

12.3 Sicurezza webhook

Il webhook ha timeout breve e non deve bloccare l'operazione principale. Se fallisce, Mirax deve loggare ma non rompere la UX.

13. RE-AUDIT

13.1 A cosa serve

I dati dei lead cambiano: siti nuovi, pixel installati, recensioni aumentate, social aggiunti. Il re-audit aggiorna lead già salvati.

13.2 Flusso

Cron /api/cron/reaudit o worker --reaudit

?

Seleziona lead vecchi in lead_pipeline

?

Rifà audit sito

?

Confronta vecchio vs nuovo

?

Se trova cambiamenti, aggiorna lead_data

13.3 Cambiamenti monitorati

- Meta Pixel installato/rimosso.
- GTM installato/rimosso.
- Instagram/Facebook aggiunti o rimossi.
- Sito creato/offline.
- Email trovata/persa.
- Rating Google variato.

14. VARIABILI AMBIENTE PER CATEGORIA

14.1 Supabase

- _PUBLIC_SUPABASE_URL
- _PUBLIC_SUPABASE_ANON_KEY
- _SERVICE_ROLE_KEY

14.2 AI

- _API_KEY

14.3 Google e Ads

- _PLACES_API_KEY
- _ADS_TOKEN

14.4 Backend

- _URL

14.5 Apollo/Snov

- _API_KEY
- _CLIENT_ID
- _CLIENT_SECRET

14.6 Pagamenti

- _SECRET_KEY
- _WEBHOOK_SECRET
- _PRICE_STARTER
- _PRICE_PRO

- _PRICE_AGENCY
- _CLIENT_ID
- _CLIENT_SECRET
- _MODE
- 14.7 Email
- _API_KEY

****Regola:**** mai mettere secret nel frontend o in PDF destinati a clienti.

15. FLUSSI COMPLETI END-TO-END

15.1 Ricerca lead

Utente su /dashboard

? compila categoria/città

POST /api/search

? crea searches.pending

Worker Python

? claim searches.processing

Google Maps + audit sito

? salva searches.results

GET /api/search/status

? UI mostra lead

15.2 Arricchimento lead

Utente apre lead detail

?

POST /api/enrich-lead

? controlla leads_cache

? se cache miss: clayEnrichLead

?? sito ufficiale

?? fonti pubbliche

?? recensioni

?? competitor

?? ads

?? social

?? trend

?? AI

?

Salva cache

?

Mostra lead completo

15.3 Salvataggio in pipeline

Lead arricchito

? click "aggiungi a pipeline"

POST /api/pipeline

? scrive lead_pipeline

/dashboard/pipeline

? visualizza kanban

Utente aggiorna stage/note/value

? PUT /api/pipeline

15.4 Insight AI

lead_pipeline contiene deal

?

/dashboard/insights

? GET /api/insights/ai

API aggrega numeri senza PII raw

?

OpenAI genera insight

?

UI mostra consigli commerciali

15.5 Billing

/dashboard/billing
? selezione piano
Stripe o PayPal
? pagamento
Webhook/capture
? aggiorna profiles
DashboardContext
? aggiorna piano e crediti

16. MAPPA INTERCONNESSIONI RAPIDA

Se tocchi Impatta

--- ---

.credits Dashboard, ricerca, billing, admin
.status Worker, polling frontend, storico ricerche
.results Tabella risultati, lead detail, export
_pipeline.lead_data Pipeline, insights, re-audit, dettaglio lead
_cache.data Velocità enrichment, costi API/OpenAI
_supabase.py Tutta la generazione lead da Google Maps
-enrichment.ts Qualità del lead detail e dati arricchiti
Piano/crediti mostrati in tutta dashboard
Stripe webhook Attivazione abbonamenti e rinnovo crediti
PayPal capture Upgrade piano via PayPal
Supabase RLS Sicurezza accesso dati utente

17. COME SPIEGARE MIRAX A UN CLIENTE TECNICO

Mirax è composto da tre livelli:

1. ****Applicazione web Next.js****: login, dashboard, UI, API serverless, billing e CRM.
2. ****Database Supabase****: conserva utenti, job, risultati, pipeline, cache e webhook.
3. ****Worker Python su VPS****: fa il lavoro pesante di scraping/audit e aggiorna Supabase.

Le API esterne sono connesse come moduli opzionali:

- Google/Maps: fonte lead e recensioni.
- OpenAI: analisi e insight.
- Apollo/Snov: contatti B2B avanzati.
- Stripe/PayPal: pagamenti.
- Resend: email.
- Webhook: invio dati verso strumenti del cliente.

Il punto centrale è Supabase: il frontend scrive richieste, i worker le leggono, i risultati tornano in Supabase, e il frontend li mostra.

18. PUNTI CRITICI DA CONOSCERE

18.1 Worker bloccati

Se un job resta troppo a lungo, può essere un worker bloccato o crashato. Serve recovery che marchi vecchi job come errore e riavvii worker.

18.2 Crediti incoerenti

Ci sono due valori storici per free: alcune UI parlano di 100 crediti, alcune inizializzazioni profilo usano 10. Va uniformato se serve coerenza prodotto.

18.3 Cache enrichment

La cache accelera molto, ma può mostrare dati non freschi fino a 90 giorni. Il re-audit compensa parzialmente.

18.4 API key mancanti

Se mancano chiavi:

- OpenAI: niente insight AI avanzati.
- Google Places: recensioni/competitor limitati.
- Apollo/Snov: niente contatti premium.
- Stripe/PayPal: billing non funziona.
- Service role: operazioni admin/cache possono fallire.

18.5 Separazione client/server

Mai importare moduli con secret o -only in componenti client.

19. GLOSSARIO RAPIDO

Termine Significato

--- ---

Lead Azienda trovata da Google Maps o inserita in pipeline

Job Riga che rappresenta una ricerca da processare

Worker Processo Python che prende job e produce risultati

Enrichment Processo di arricchimento dati del lead

Pipeline CRM interno con fasi commerciali

Opportunity score Punteggio di potenziale commerciale

Freshness score Quanto è recente l'audit

RLS Row Level Security Supabase

Service role Chiave server Supabase con permessi elevati

Webhook Invio automatico evento a URL esterno

20. CONCLUSIONE

Mirax funziona come una piattaforma a coda: il frontend crea richieste, Supabase le conserva, i worker Python le processano, i risultati tornano nel database, il frontend li mostra e li trasforma in pipeline CRM. Gli arricchimenti e le analisi AI aggiungono valore commerciale al dato grezzo di Google Maps. Billing, crediti, webhook e insights completano il prodotto SaaS.

Questo documento descrive ogni blocco principale, a cosa serve, chi lo usa e come si collega agli altri elementi.